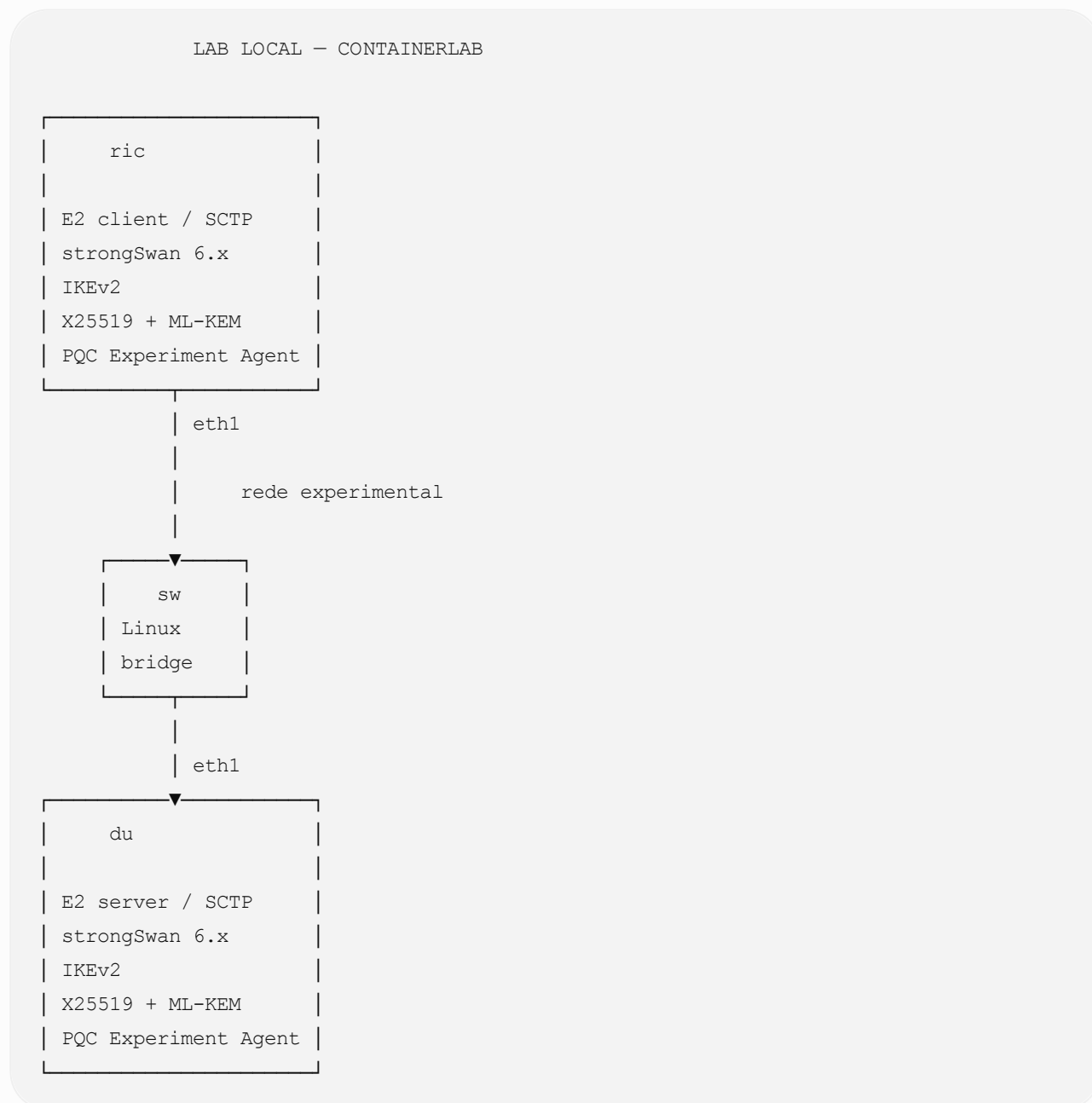


Sim. **Containerlab** é uma boa escolha para pré-montar essa arquitetura antes do **testbed**, principalmente para validar topologia, configuração do strongSwan, negociação IKEv2, criação das SAs, coleta de métricas e automação. O Containerlab é justamente voltado a montar laboratórios de rede baseados em containers e permite usar containers Linux genéricos como endpoints/geradores de tráfego.  Containerlab +1

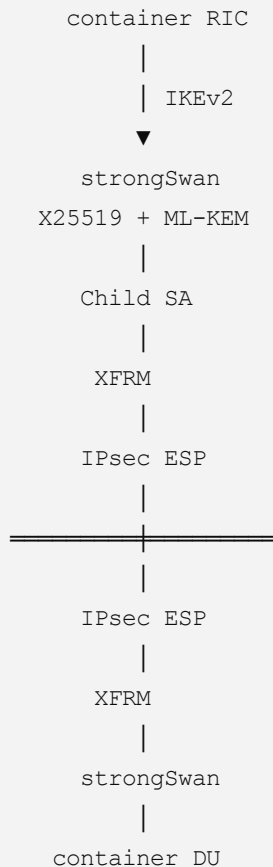
Eu montaria primeiro uma versão reduzida da ilha:



Containerlab consegue ligar os containers diretamente ou através de um **Linux bridge**, então esse desenho é perfeitamente compatível com o modelo da ferramenta.  Containerlab...

O que você já consegue validar localmente

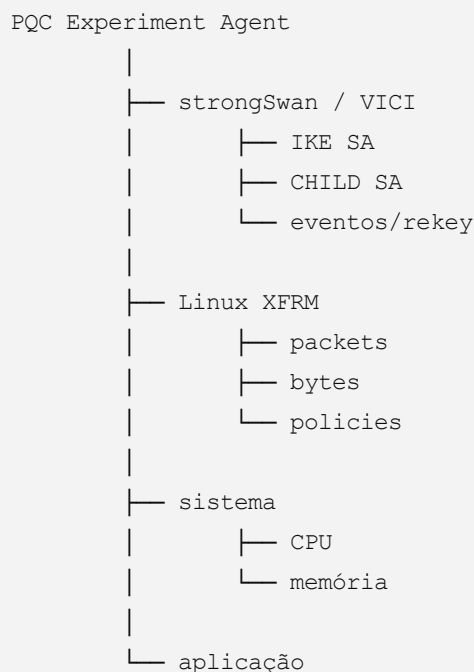
A parte mais importante é que você consegue chegar ao testbed com praticamente todo o **plano de segurança** pronto:



Você pode validar se o IKEv2 sobe, se o ML-KEM foi realmente negociado, se a `IKE SA` e a `CHILD SA` aparecem corretamente, se o tráfego escolhido está entrando em ESP e se o rekey funciona. O strongSwan usa o IPsec do kernel Linux normalmente, e sua documentação trata XFRM como uma das formas de implementar esse dataplane.

 StrongSwan Do... +1

Também dá para implementar o **PQC Experiment Agent** quase inteiro antes de chegar à RNP:



- └─ RTT
- └─ jitter
- └─ throughput

Então quando forem para a ilha, vocês não começam desenvolvendo. Começam **portando uma arquitetura já testada.**

Mas existe uma pegadinha importante: IPsec dentro de container

Esse é o ponto que eu testaria primeiro.

O `charon` roda em userspace, mas o IPsec/XFRM tradicional é implementado pelo **kernel Linux**. Containers compartilham o kernel do host. Portanto, o container precisa receber permissões suficientes para operar a pilha de rede/XFRM correspondente.

Então um container Linux completamente restrito pode não conseguir simplesmente fazer:

```
ip xfrm state
ip xfrm policy
```

e instalar as SAs necessárias.

Essa também é uma das razões pelas quais implementações de strongSwan em containers frequentemente usam privilégios de rede especiais ou networking do host. O Containerlab suporta explicitamente `network-mode: host`, embora eu não começaria assim porque perde parte do isolamento da topologia.  Containerlab +1

Eu tentaria primeiro:

```
container dedicado
+
network namespace próprio
+
NET_ADMIN
+
XFRM no namespace
```

antes de recorrer a `host networking`.

Topologia que eu usaria

Algo próximo disto:

YAML

```

name: openran-pqc

topology:
  nodes:

    ric:
      kind: linux
      image: openran-pqc:latest

    du:
      kind: linux
      image: openran-pqc:latest

  links:
    - endpoints:
        - "ric:eth1"
        - "du:eth1"

```

O próprio Containerlab trata containers Linux comuns como `kind: linux` e cria a conectividade entre as interfaces da topologia.  Containerl...

A imagem `openran-pqc` poderia conter:

```

Ubuntu/Debian
├──
├── strongSwan 6.x
├──
├── swanctl
├── iproute2
├── tcpdump
├── iperf3
├── ferramentas SCTP
├── Python
└── PQC Experiment Agent

```

Uma vantagem grande é que essa mesma imagem pode depois servir de referência para preparar as VMs/hosts na RNP.

Eu não colocaria o Near-RT RIC real logo de início

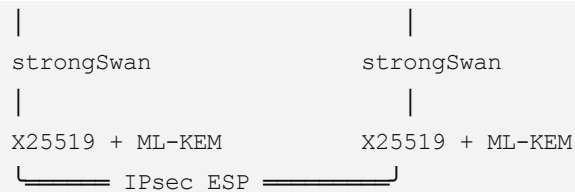
Eu faria em camadas.

Fase local 0 — validar apenas IPsec/PQC

```

ric                                du
|                                 |
ping / iperf                      server

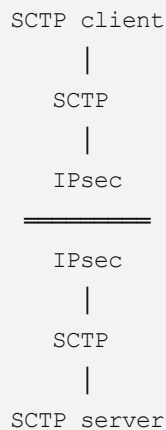
```



Objetivo:

provar que a infraestrutura PQC funciona.

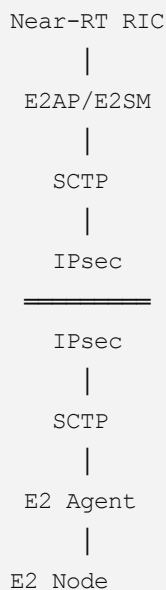
Fase local 1 — adicionar SCTP



Isso começa a aproximar o ambiente da E2.

Fase local 2 — E2 real

Se vocês tiverem acesso às implementações necessárias:



Assim, se alguma coisa quebrar, você sabe **em qual camada** quebrou.

O Containerlab também ajuda muito na Etapa 2

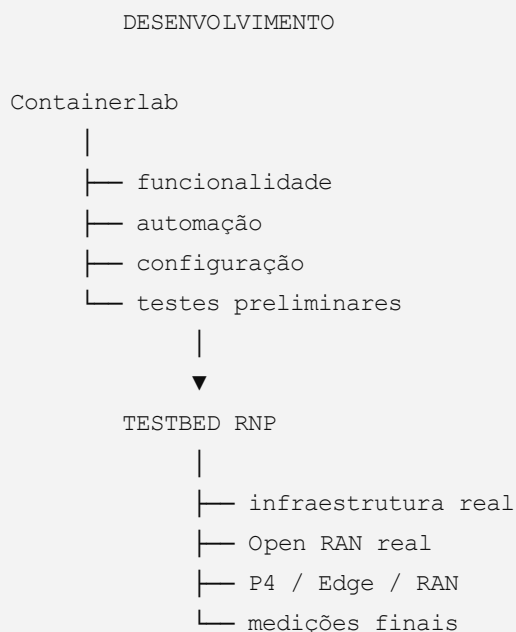
Aquela etapa do documento em que introduzimos condições adversas de rede também pode ser parcialmente reproduzida localmente.

Você pode deixar conceitualmente:



No laboratório local isso pode ser implementado com recursos de Linux networking sobre os enlaces; depois, na RNP, vocês podem avaliar se vale substituir essa emulação pelo domínio P4 do testbed.

Então teríamos:



Isso é uma narrativa ótima também para a reunião.

E você consegue fazer os três cenários no mesmo YAML

Por exemplo, podemos ter uma variável do experimento:

```
MODE=baseline
MODE=ipsec-classical
```

MODE=ipsec-pqc

E seu agente/controlador configura:

```
baseline
  ↓
nenhuma SA

classical
  ↓
IKEv2
X25519
  ↓
ESP

pqc
  ↓
IKEv2
X25519 + ML-KEM-768
  ↓
ESP
```

Então o fluxo de execução poderia ser:

```
containerlab deploy
  |
  ▼
configura IPs
  |
  ▼
seleciona cenário
  |
  ▼
inicia strongSwan
  |
  ▼
valida IKE/CHILD SA
  |
  ▼
gera tráfego
  |
  ▼
coleta métricas
  |
  ▼
destroy
```

O Containerlab cuida justamente do ciclo de vida e da montagem da topologia; isso combina muito bem com experimentos reproduzíveis.  Containerl...

O que não devemos concluir a partir do Containerlab

As medições locais não devem ser tratadas como resultado final de desempenho da Open RAN.

Por exemplo:

```
Containerlab:
ML-KEM adicionou X ms
```

não significa automaticamente:

```
RNP:
ML-KEM adicionará X ms
```

porque os containers compartilham recursos do host, o caminho de rede é virtualizado e a topologia física é completamente diferente.

Eu usaria os resultados locais como:

validação funcional e caracterização preliminar.

E os resultados do OpenRAN@Brasil como:

avaliação experimental no testbed.

Isso inclusive fortalece a metodologia.

Eu acho que vale bastante montarmos esse laboratório agora. O próximo passo poderia ser definir a topologia `openran-pqc.clab.yml`, a imagem Docker com **strongSwan 6.x + ML-KEM**, os endereços das interfaces e os três modos de experimento. A partir daí, você chegaria à reunião da RNP não apenas com um diagrama, mas podendo dizer: **“já temos um protótipo reproduzível da arquitetura funcionando em Containerlab; precisamos mapear esses dois endpoints para os recursos do PoP-RJ.”**